UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA EN SOFTWARE

INFORME DEL EJERCICIO 9: CONTROL DEL MODO DE APERTURA (APPEND VS SOBRESCRIBIR)

T2.4Fase1APE5. Creación, Lectura y Escritura de Ficheros · Tercera parte

Dato	Detalle
Asignatura	Algoritmos y Lógica de Programación
Nivel y paralelo	Primero – B
Estudiante	Paredes Acosta Dereck Shair
Docente	Ing. José Ruben Caizabuano, M.Sc.
Lenguaje	Java

1. Objetivo

Demostrar el comportamiento del segundo parámetro del constructor de FileWriter, agregando dos asignaturas a materias.txt sin perder su contenido.

2. Enunciado

A partir del archivo "materias.txt" generado en el ejercicio anterior, escribe un nuevo programa en Java que demuestre el comportamiento del segundo parámetro en el constructor de FileWriter:

1. Diseña el código de manera que abra el fichero existente configurando el FileWriter en modo de adición (append = true).

2. Añade dos nuevas asignaturas al final del archivo utilizando nuevamente newLine() para no romper el formato de líneas.

3. Asegúrate de cerrar el flujo con .close().

4. Comprobación: Verifica que el contenido previo no se haya sobrescribido y que ahora el archivo conserve las cinco asignaturas originales junto con las dos nuevas incorporadas al final.

3. Análisis y diseño de clases

Clase	Objetos o variables	Métodos utilizados
BufferedWriter	escritor: BufferedWriter	write(texto); newLine(); close()
FileWriter	archivo: FileWriter	FileWriter(nombre, true)
IOException	e: IOException	Excepción capturada en el bloque catch
Clase principal	—	main(String[] args)

4. Código en Java

Archivo: Ejercicio9ModoAppend.java

```
// Ejercicio 9: Control del modo de apertura (append vs sobrescribir)
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio9ModoAppend {

    public static void main(String[] args) {
        try {
            // 1. El segundo parámetro en true activa el modo de adición
            FileWriter archivo = new FileWriter("materias.txt", true);
            BufferedWriter escritor = new BufferedWriter(archivo);

            // 2. Las nuevas asignaturas se agregan al final
            escritor.write("Desarrollo Web");
            escritor.newLine();
            escritor.write("Ciberseguridad");
```

```
            escritor.newLine();

            // 3. Cierre del flujo
            escritor.close();
            System.out.println("Se agregaron 2 asignaturas a materias.txt.");
        } catch (IOException e) {
            System.out.println("Se produjo un error al escribir en el archivo.");
        }
    }
}
```

5. Prueba de escritorio

Paso	Instrucción	Estado de variables / memoria	Salida o resultado
1	Estado inicial	materias.txt tiene 5 líneas	—
2	new FileWriter("materias.txt", true)	Se abre en modo de adición; las 5 líneas se conservan	—
3	escritor.write("Desarrollo Web"); newLine()	Se agrega la línea 6	—
4	escritor.write("Ciberseguridad"); newLine()	Se agrega la línea 7	—
5	escritor.close(); System.out.println(...)	Se guarda y se cierra; materias.txt tiene 7 líneas	Se agregaron 2 asignaturas a materias.txt.

6. Resultado de la ejecución

El programa se ejecutó una vez, después del Ejercicio 8. El archivo conserva las cinco asignaturas originales y las dos nuevas al final:

```
Se agregaron 2 asignaturas a materias.txt.

Contenido de materias.txt:
Algoritmos
Sistemas Operativos
Base de Datos
Redes de Computadoras
Estructura de Datos
Desarrollo Web
Ciberseguridad
```

7. Conclusión

Con el segundo parámetro de FileWriter en true, las nuevas asignaturas se agregan al final y el contenido previo se conserva, mientras que sin él el archivo se sobrescribe. Usar newLine() mantiene el formato de una asignatura por línea.